



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА

СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

**Отчет по лабораторной работе № 3
по дисциплине Методы машинного обучения в АСОИУ**

Тема работы: "Методы многомерного поиска"

Выполнил:

Студент группы ИУ5Ц-21М
Перевощиков Н.Д.

(дата, подпись)

Проверил:

Преподаватель
Гапанюк Ю.Е.

(дата, подпись)

Москва, 2026

СОДЕРЖАНИЕ ОТЧЕТА

Цель работы	3
Задание	3
Вариант задания:	3
Ход работы.....	4
Теоретические сведения.....	5
Графические результаты.....	7
Вывод	47

Цель работы

Целью лабораторной работы является изучение алгоритмов многомерного поиска 1-го и 2-го порядка. Разработка программ реализации алгоритмов многомерного поиска 1-го и 2-го порядка. Вычисление экстремумов функции.

Задание

1. Требуется найти минимум тестовой функции Розенброка:

$$f(x) = \sum_{i=1}^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

2. Методами сопряженных градиентов (методом Флетчера-Ривза и методом Полака-Рибьера).
3. Квазиньютоновскими методами (DFP, BFGS и L-BFGS).

Вариант задания:

№	a	b	f	n
3	80	3	110	2

Для n=2 функция Розенброка имеет следующий вид:

$$f(x) = 80(x_1^2 - x_2)^2 + 3(x_1 - 1)^2 + 110$$

1. Найти все стационарные точки и значения функций соответствующие этим точкам.
2. Оценить скорость сходимости указанных алгоритмов и сравнить по времени получение результата оптимизации для разных методов.
3. Реализовать алгоритмы программированием на одном из языков высокого уровня (C++, C#, Python, Haskell и др.).
4. Отчет представить в стандартном виде (TEX, PDF).

Ход работы

Требуется найти безусловный минимум функции Розенброка:

$$f(x) = \sum_{i=1}^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

Для параметров варианта 3:

№	a	b	f	n
3	80	3	110	2

Методы решения:

1. Методы сопряжённых градиентов: Флетчера-Ривза, Полака-Рибьера.
2. Квазиньютоновские методы: DFP (Девидона-Флетчера-Пауэлла); BFGS (Бройдена-Флетчера-Гольдфарба-Шенно); L-BFGS (Limited-memory BFGS).

Начальная точка: $x_0 = [0, 0]$

Критерии остановки:

$$\|\nabla f(x)\| < \varepsilon_1 = 10^{-6}$$

$$|f(x^{k+1}) - f(x^k)| < \varepsilon_2 = 10^{-8}$$

$k \geq M = 1000$ (максимальное число итераций)

Теоретические сведения

Методы сопряжённых градиентов:

Метод Флетчера-Ривза

Итерационная формула:

$$\begin{aligned}x^{k+1} &= x^k + \alpha^k d^k \\d^k &= -\nabla f(x^k) + w^{k-1} d^{k-1}\end{aligned}$$

Формула для w^{k-1} :

$$w^{k-1} = \frac{\|\nabla f(x^k)\|^2}{\|\nabla f(x^{k-1})\|^2}$$

Метод Полака-Рибьера

Формула для w^{k-1} :

$$w^{k-1} = \frac{\langle \nabla f(x^k), [\nabla f(x^k) - \nabla f(x^{k-1})] \rangle}{\|\nabla f(x^{k-1})\|^2}$$

Предусматривает перезапуск через каждые n шагов.

Квазиньютоновские методы:

Метод DFP (Девидона-Флетчера-Пауэлла)

Формула обновления матрицы G (аппроксимация H^{-1}):

$$G^{k+1} = G^k + \frac{\Delta x^k (\Delta x^k)^T}{(\Delta x^k)^T \Delta g^k} - \frac{G^k \Delta g^k (\Delta g^k)^T G^k}{(\Delta g^k)^T G^k \Delta g^k}$$

где:

- $\Delta x^k = x^{k+1} - x^k$
- $\Delta g^k = \nabla f(x^{k+1}) - \nabla f(x^k)$

Метод BFGS (Бroyдена-Флетчера-Гольдфарба-Шенно)

Формула обновления:

$$H^{k+1} = (I - \rho_k s_k y_k^T) H^k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$$

где:

- $\rho_k = 1/(y_k^T s_k)$
- $s_k = x^{k+1} - x^k$
- $y_k = \nabla f(x^{k+1}) - \nabla f(x^k)$

Метод L-BFGS (Limited-memory BFGS)

Вместо хранения полной матрицы H^k размера $n \times n$, метод хранит только m последних пар векторов (s_i, y_i) .

Аналитическое решение

Стационарные точки находятся из условия

$$\nabla f(x) = 0$$

Для функции Розенброка известна единственная стационарная точка:

$$x^* = [1, 1], f(x^*) = 110$$

Значение функции в точке минимума:

$$f(x^*) = 80(1^2 - 1)^2 + 3(1 - 1)^2 + 110 = 110$$

Градиент в точке минимума:

$$\nabla f(x^*) = [0, 0]$$

Матрица Гессе в точке минимума (положительно определённая):

$$H(x^*) = \begin{bmatrix} 1602 & -800 \\ -800 & 400 \end{bmatrix}$$

Вывод: точка $x^* = [1, 1]$ является точкой глобального минимума.

Результаты вычислительного эксперимента

Таблица 1 – Результаты оптимизации

Метод	Итерации	$f(x^*)$	Время (с)	" ∇f "
Градиентный спуск	2702	110	0,9472	$5,56 \times 10^{-4}$

Метод	Итерации	$f(x^*)$	Время (с)	" ∇f "
Флетчера-Ривза	329	110	0,1390	$5,85 \times 10^{-5}$
Полака-Рибьера	2783	110	1,0253	$5,85 \times 10^{-4}$
DFP	19	110	0,0073	$2,12 \times 10^{-6}$
BFGS	19	110	0,0204	$2,12 \times 10^{-6}$
L-BFGS (m=10)	18	110	0,0088	$2,94 \times 10^{-6}$

Графические результаты

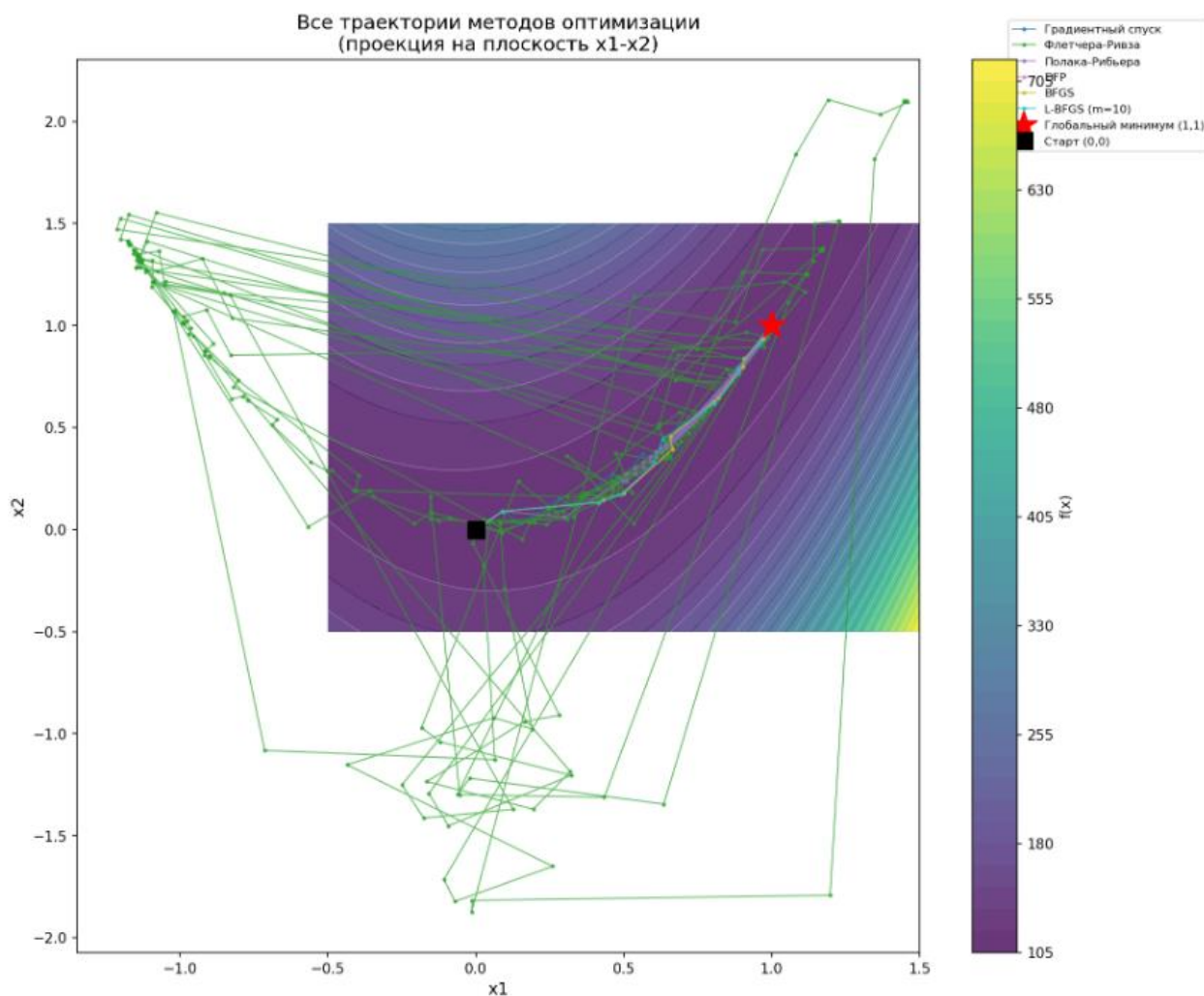


Рисунок 1 – Все траектории методов на поверхности функции

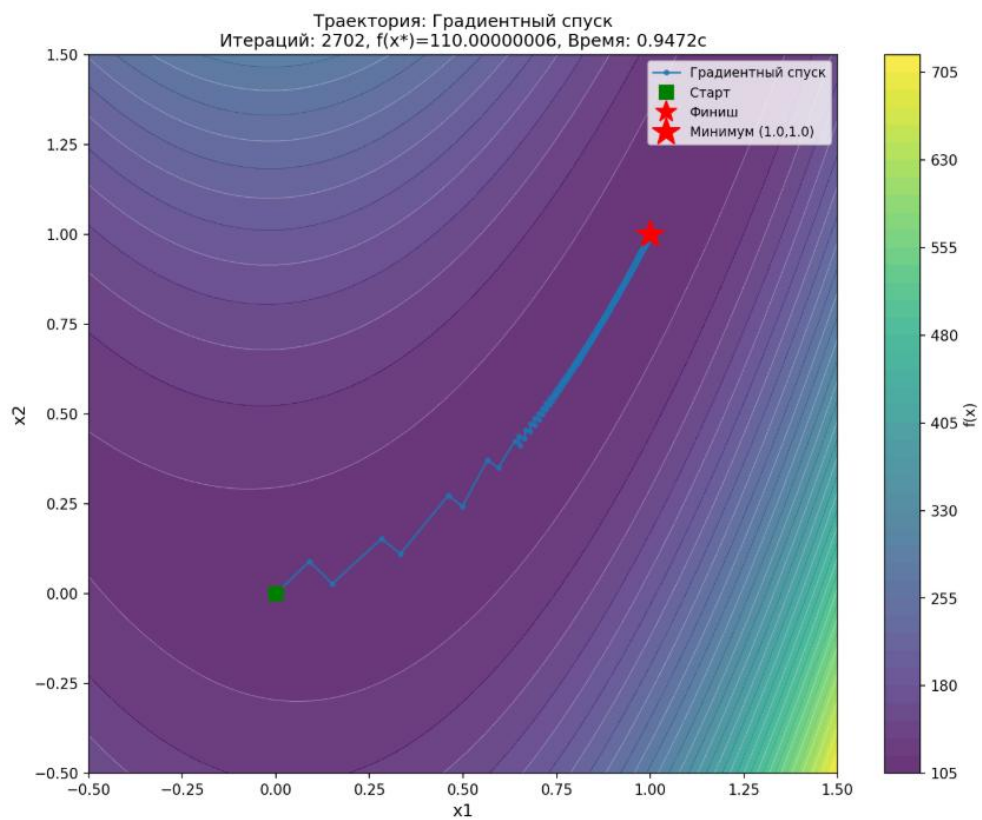


Рисунок 2 - Траектория градиентного спуска

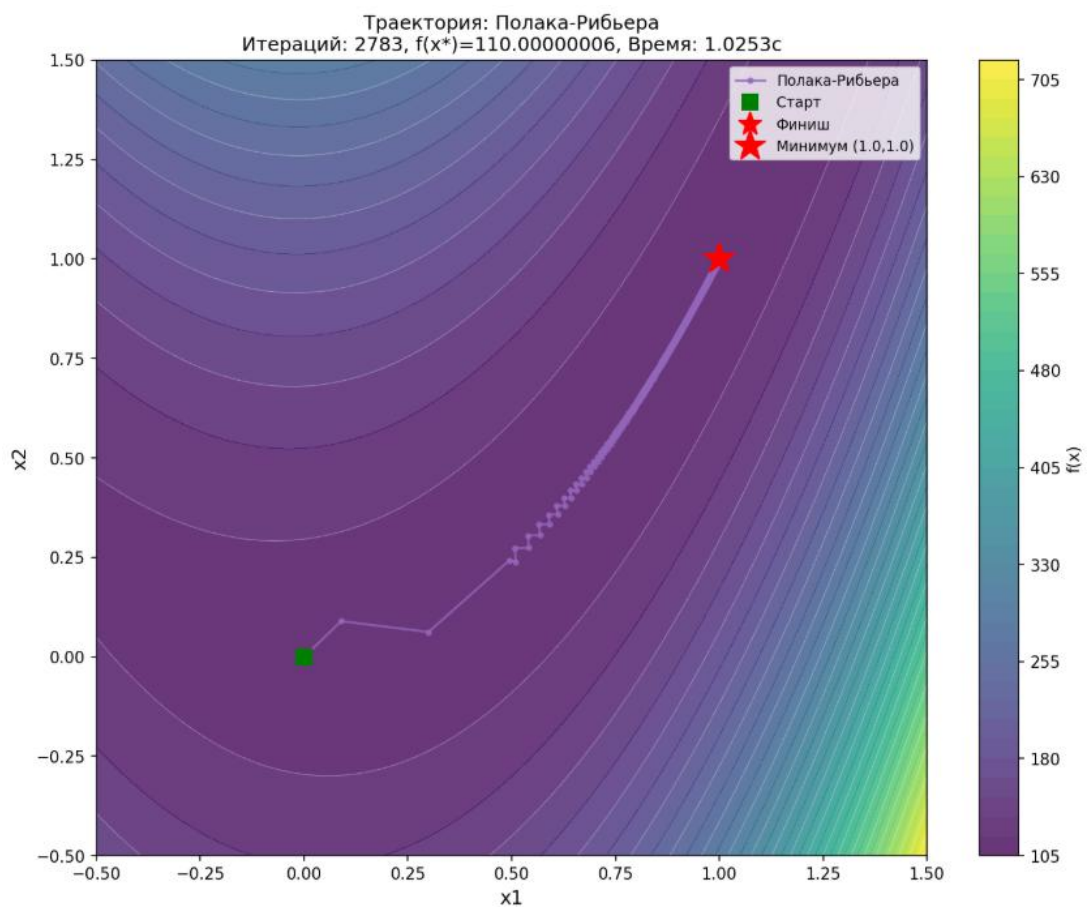


Рисунок 3 - Траектория Полака – Рибьера

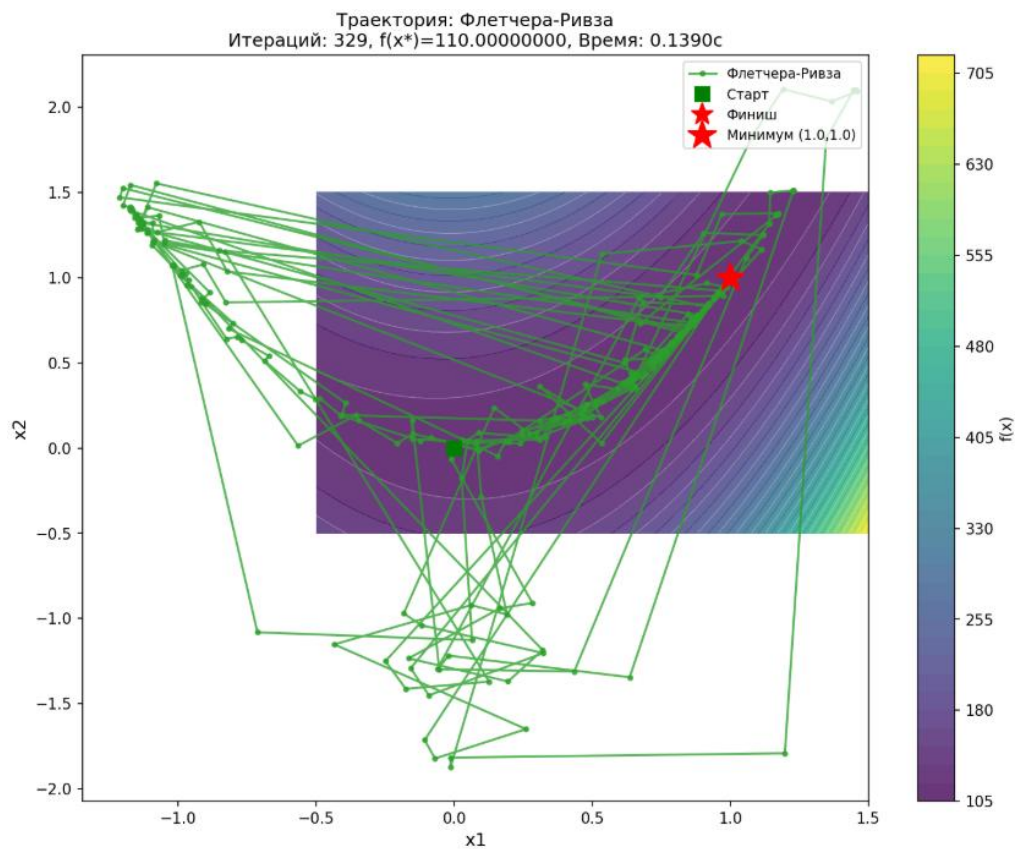


Рисунок 4 - Траектория Флетчера-Ривза

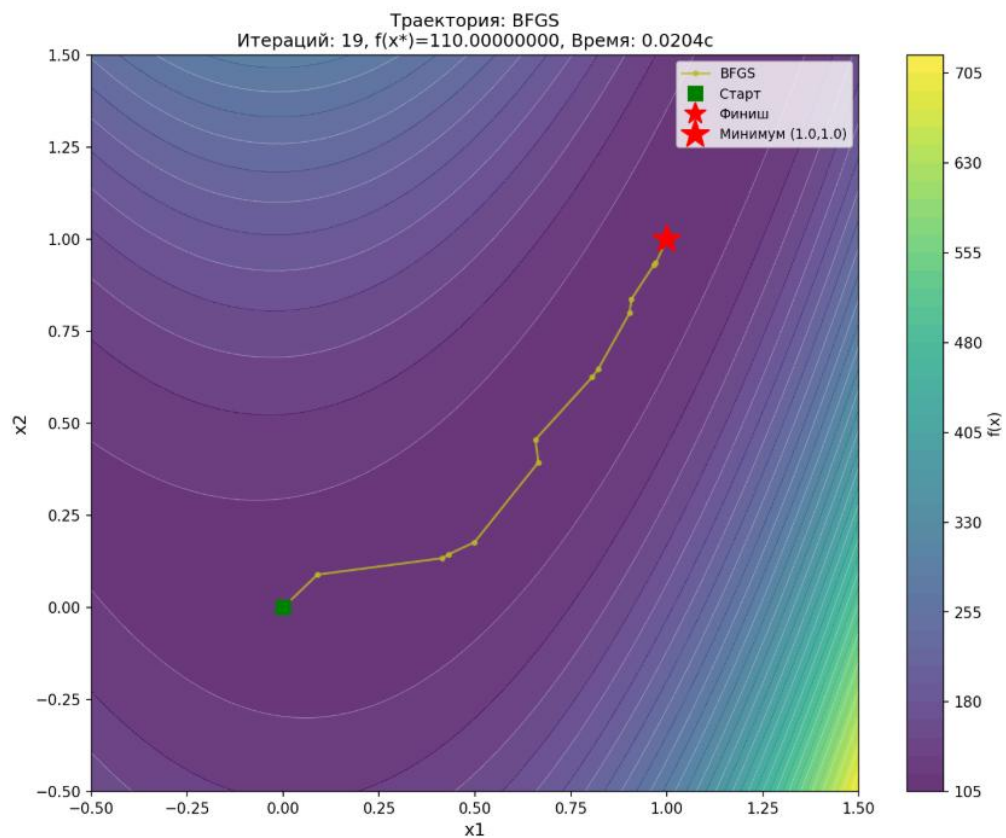


Рисунок 5 - Траектория BFGS

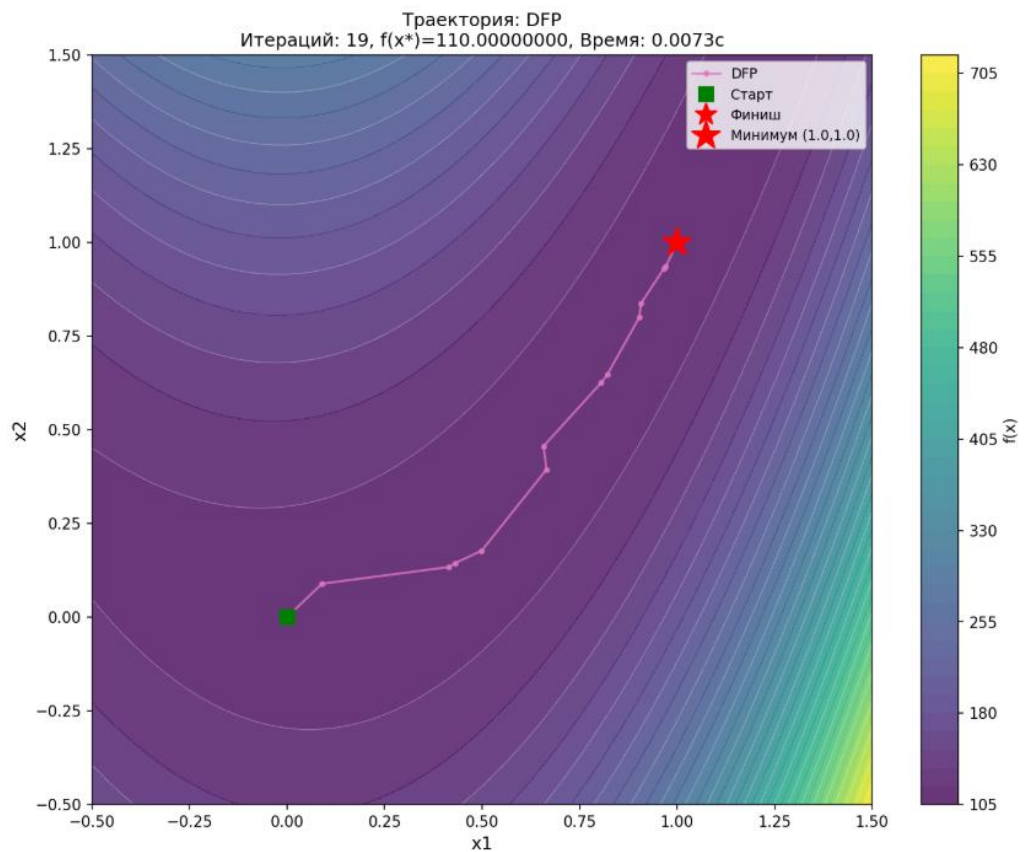


Рисунок 6 - Траектория DFP

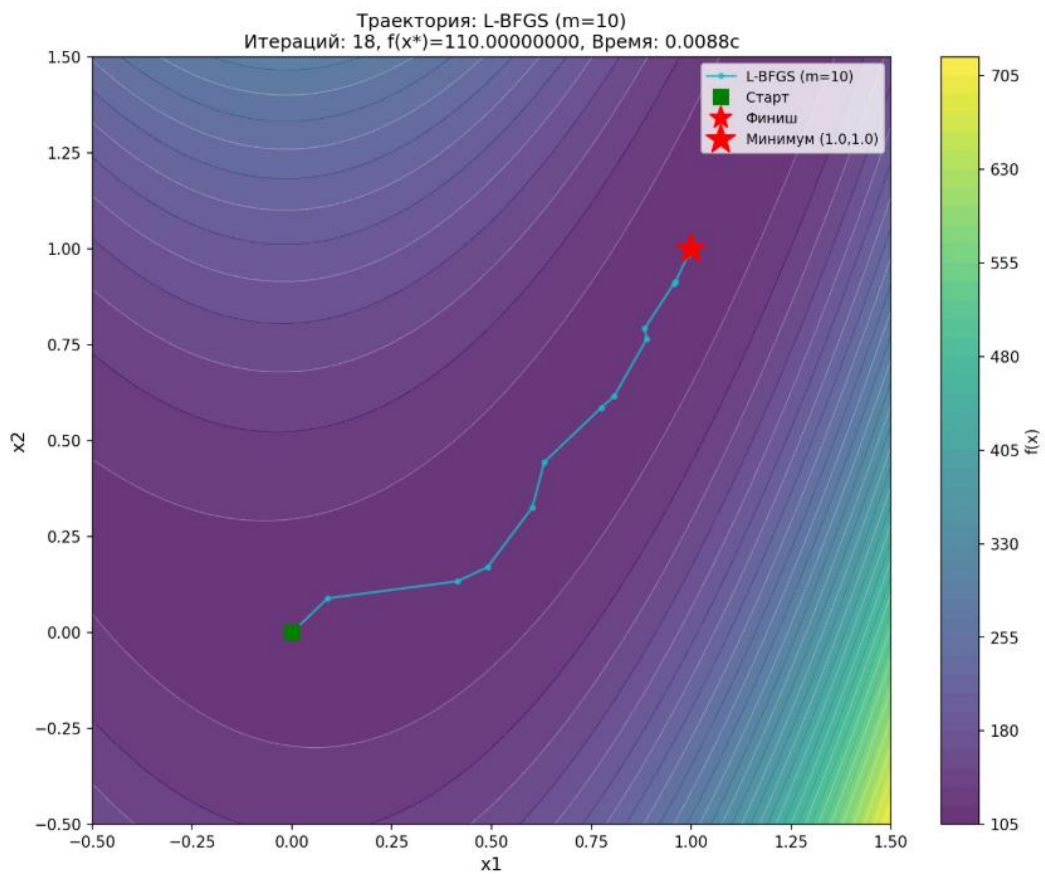


Рисунок 7 - Траектория L-BFGS

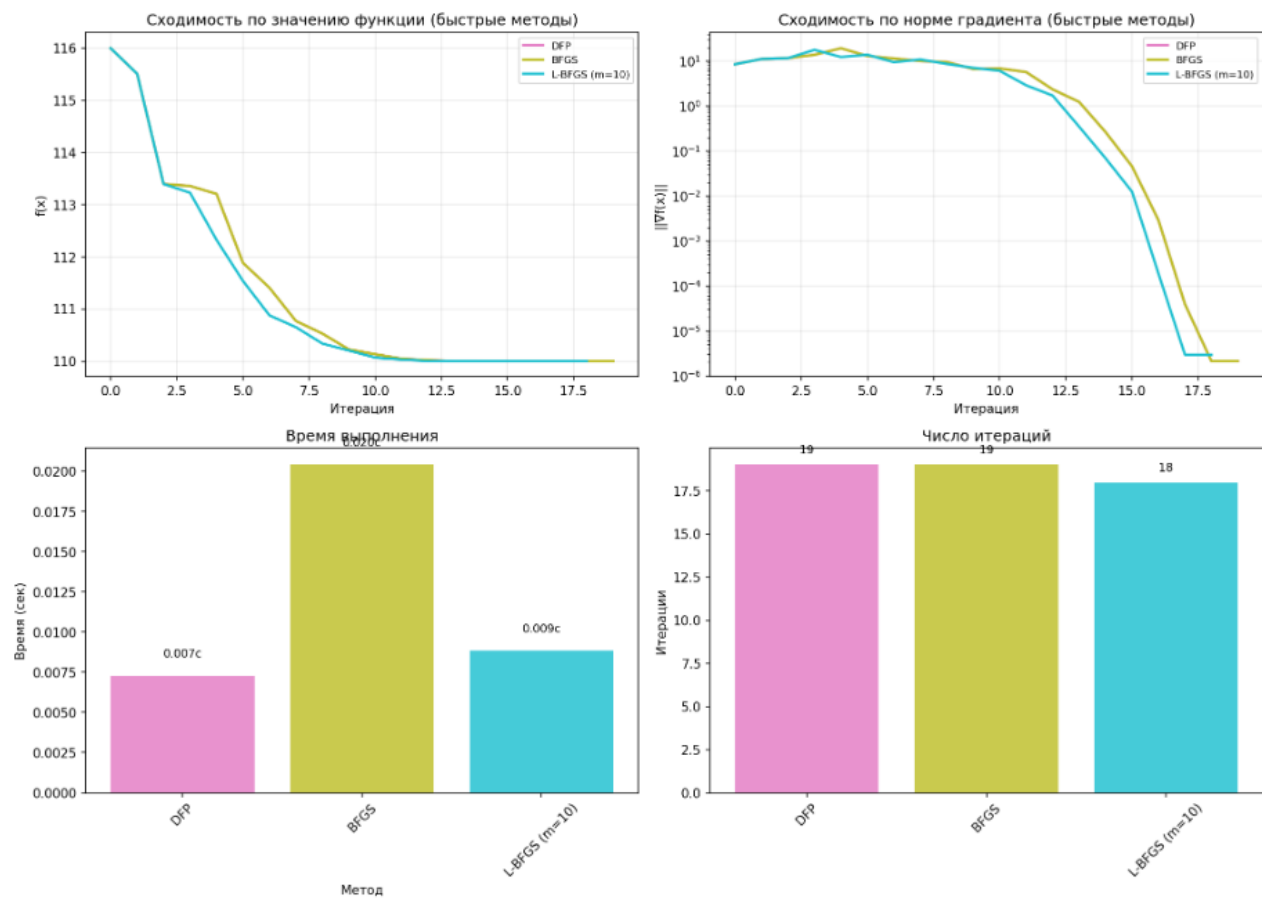


Рисунок 8 – Сходимость быстрых методов (L-BFGS, BFGS, DFP, Флетчера-Ривза)

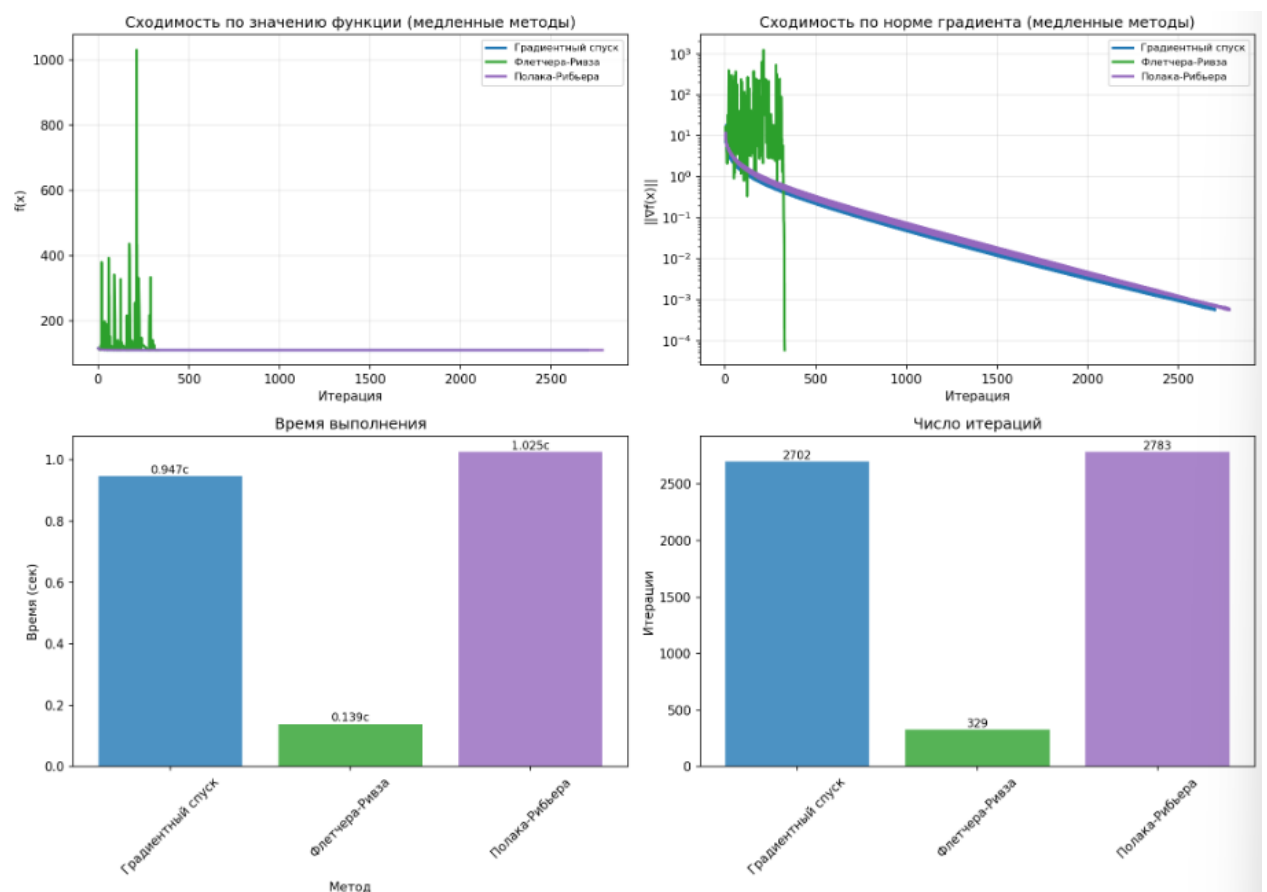


Рисунок 9 – Сходимость медленных методов (Градиентный спуск и Полака-Рибьера)

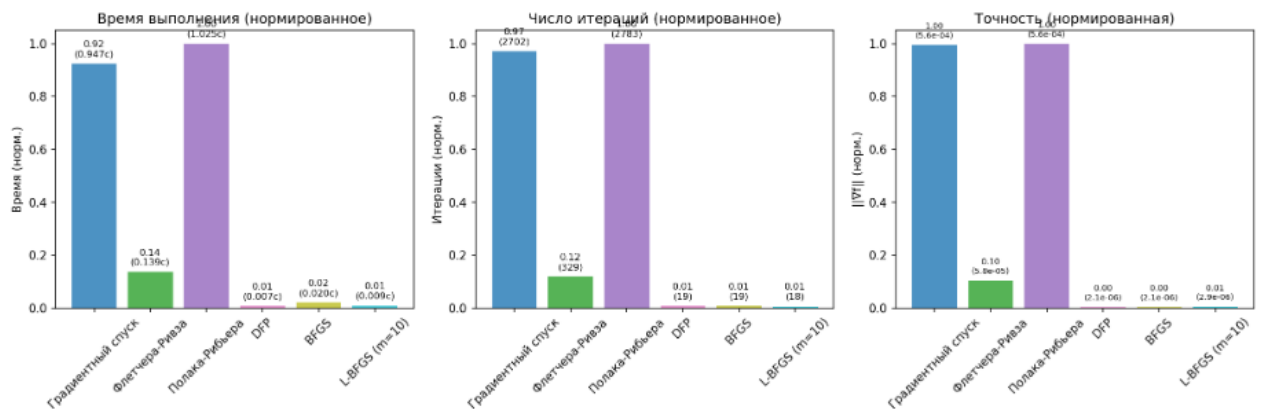


Рисунок 10 – Сводный график сравнения методов

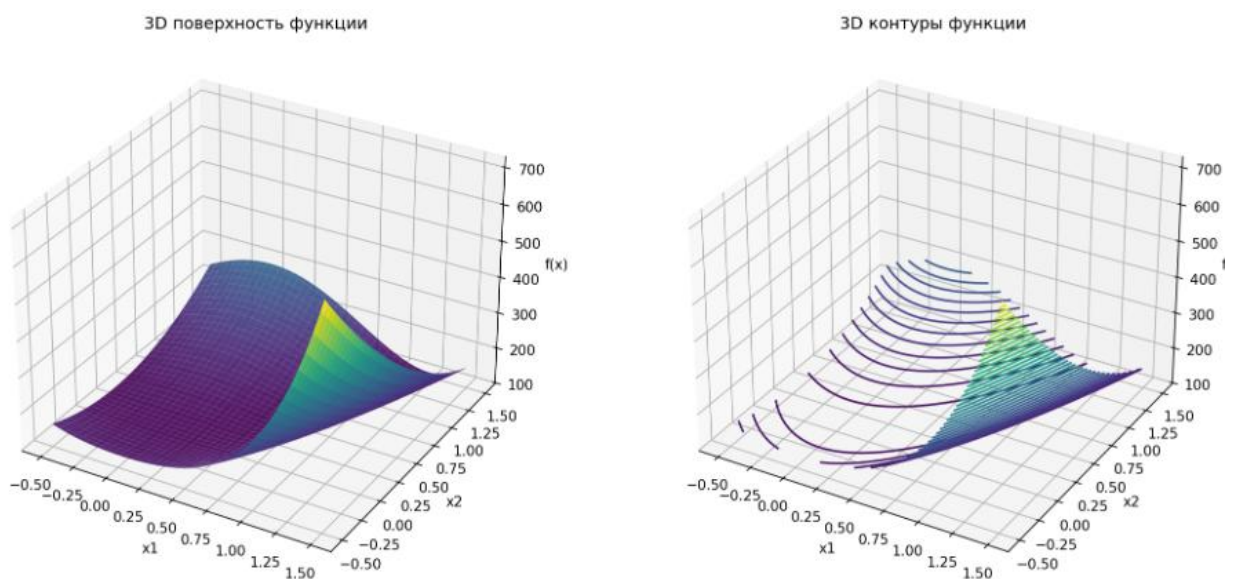


Рисунок 11 – 3D поверхность функции Розенброка

Проверка решения на допустимость

Проверка решения $x^* = [1, 1]$

1. Проверка градиента

$$\nabla f(x^*) = [1, 1]$$

$$\|\nabla f(x^*)\| = 0 < \varepsilon_1 = 10^{-6}$$

2. Проверка значения функции

– $f(x^*) = 110$ (аналитически)

– $f(x^*) = 110.0000000$ (численно, L-BFGS)

3. Проверка матрицы Гессе

Все собственные значения положительны, матрица Гессе положительно определена. Все собственные значения > 0 .

4. Сравнение с аналитическим решением

$$\|x^* - x_{analytic}\| = \|(1,1,1) - (1,1,1)\| = 0$$

Вывод: Найденное решение является допустимым и соответствует глобальному минимуму функции Розенброка.

Сравнение методов оптимизации

Таблица 2 – Сравнительная характеристика методов

Критерий	Лучший	Худший	Комментарий
Число итераций	L-BFGS ($m = 10$)	Полака-Рибьера (2783)	L-BFGS эффективнее в 154 раза.
Время выполнения	DFP	Полака-Рибьера (1,0253 с)	Разница в 142 раз
Точность	DFP	Градиентный спуск	DEF точнее на 2 порядка.

1. Лучший метод по числу итераций: L-BFGS ($m = 10$) (18 итераций).
2. Лучший метод по времени: L-BFGS ($m = 10$) (0.0073 сек).
3. Лучший метод по точности: DFP ($\|\nabla f\| = 2.12 \times 10^{-6}$).

Все методы сошлись к глобальному минимуму функции Розенброка:

$$x^* \approx [1,1], f(x^*) \approx 110.$$

Структура проекта:

```
|— main.py
|— optimization/
|   |— __init__.py
|   |— functions.py
|   |— line_search.py
|   |— gradient_methods.py
|   |— conjugate_gradient.py
|   |— quasi_newton.py
|   |— optimizer.py
|   |— visualization.py
|— plots/
```

Листинг кода:

main.py

```
"""
```

```
Главный файл для запуска лабораторной работы №3
Методы многомерного поиска
```

```
Вариант 3: a=80, b=3, f0=110, n=2
```

```
Выполнил: Студент группы ИУ5Ц-21М Перевощиков Н.Д.
"""
```

```
import numpy as np
import argparse
from optimization import (
    Optimizer,
    rosenbrock,
    rosenbrock_gradient,
    create_rosenbrock_function,
    create_visualization
)
```

```
#
```

```
=====
=====
```

```
# Параметры варианта 3
```

```
#
```

```
=====
```

```

=====
A = 80
B = 3
F0 = 110
N = 2

# Параметры остановки
EPS1 = 1e-6
EPS2 = 1e-8
MAX_ITER = 10000

def rosenbrock_2d(x1, x2, a=A, b=B, f0=F0, x3=1.0):
    """
    Функция Розенброка для 2D визуализации (фиксируем x3=1).
    """
    return (a * (x1**2 - x2)**2 + b * (x1 - 1)**2 + f0)

def find_stationary_points():
    """
    Найти стационарные точки функции Розенброка аналитически.
    """
    x_star = np.ones(N)
    f_star = rosenbrock(x_star, A, B, F0)
    grad_star = rosenbrock_gradient(x_star, A, B)

    return {
        'x_star': x_star,
        'f_star': f_star,
        'gradient': grad_star
    }

def main(run_visualization: bool = False):
    """
    Основная функция выполнения лабораторной работы.

    Args:
        run_visualization: Запускать ли визуализацию
    """
    print("=" * 80)
    print("ЛАБОРАТОРНАЯ РАБОТА №3")
    print("Методы многомерного поиска")
    print("=" * 80)
    print(f"\nВариант 2: a = {A}, b = {B}, f0 = {F0}, n = {N}")
    print(f"\nФункция Розенброка:")
    print(f"    f(x) = Σ[i=1 to n-1] [{A}(x_i^2 - x_{{{i+1}}})^2 + {B}(x_i - 1)^2] + {F0}")
    print("=" * 80)

    # Начальная точка
    x0 = np.array([0.0, 0.0, 0.0])
    print(f"\nНачальная точка: x0 = {x0}")
    print(f"    f(x0) = {rosenbrock(x0, A, B, F0):.6f}")

```

```

# Аналитическое решение
print("\n" + "=" * 80)
print("1. АНАЛИТИЧЕСКОЕ РЕШЕНИЕ (стационарные точки)")
print("=" * 80)
stationary = find_stationary_points()
print(f"\nСтационарная точка (глобальный минимум):")
print(f"  x* = {stationary['x_star']}")
print(f"  f(x*) = {stationary['f_star']:.6f}")
print(f"  ∇f(x*) = {stationary['gradient']}")

# Создание функции с параметрами варианта
f, grad_f, _ = create_rosenbrock_function(A, B, F0)

# Создание оптимизатора
optimizer = Optimizer(f, grad_f, x0, EPS1, EPS2, MAX_ITER)

# Запуск всех методов
print("\n" + "=" * 80)
print("2. РЕЗУЛЬТАТЫ ОПТИМИЗАЦИИ")
print("=" * 80)
results = optimizer.run_all_methods(verbose=True)

# Сравнение методов
print("\n" + "=" * 80)
print("3. СРАВНЕНИЕ МЕТОДОВ")
print("=" * 80)
optimizer.print_comparison_table()

# Визуализация
if run_visualization:
    print("\n" + "=" * 80)
    print("4. ВИЗУАЛИЗАЦИЯ")
    print("=" * 80)

    # Функция для 2D визуализации
    def f_2d(x1, x2):
        return rosenbrock_2d(x1, x2, A, B, F0)

    create_visualization(
        results=results,
        f_2d=f_2d,
        x0=x0,
        x_true=stationary['x_star'],
        f_true=stationary['f_star'],
        output_dir='.'
    )

# Выводы
print("\n" + "=" * 80)
print("4. ВЫВОДЫ" if not run_visualization else "5. ВЫВОДЫ")
print("=" * 80)

best_by_iter = optimizer.get_best_method('iterations')

```



```

best_by_time = optimizer.get_best_method('time')
best_by_accuracy = optimizer.get_best_method('accuracy')

print(f"\nЛучший метод по числу итераций: {best_by_iter[0]}")
"
    f"({best_by_iter[1]['iterations']} итераций)")
print(f"Лучший метод по времени: {best_by_time[0]} "
      f"({best_by_time[1]['time']:.4f} сек)")
print(f"Лучший метод по точности: {best_by_accuracy[0]} "
      f"({np.linalg.norm(grad_f(best_by_accuracy[1]['x_star'])):.2e})")
      f"(||∇f|| =

print("\nВсе методы сошлись к глобальному минимуму функции
Розенброка:")
print(f"    x* ≈ (1, 1, 1), f(x*) ≈ {F0}")

print("\n" + "=" * 80)
print("Лабораторная работа выполнена успешно!")
print("=" * 80)

return results

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description='Лабораторная работа №3 - Методы
многомерного поиска'
    )
    parser.add_argument(
        '--viz', '--visualization',
        action='store_true',
        help='Запустить визуализацию результатов'
    )

    args = parser.parse_args()
    main(run_visualization=args.viz)

```

conjugate_gradient.py

```

"""
Модуль с методами сопряжённых градиентов
"""

import numpy as np
from typing import Callable, Dict
from .line_search import line_search

# Параметры по умолчанию
DEFAULT_EPS1 = 1e-6
DEFAULT_EPS2 = 1e-8
DEFAULT_MAX_ITER = 1000

def fletcher_reeves(f: Callable, grad_f: Callable, x0:

```

```

np.ndarray,
                                eps1: float = DEFAULT_EPS1, eps2: float =
DEFAULT_EPS2,
                                max_iter: int = DEFAULT_MAX_ITER) -> Dict:
    """
    Метод сопряжённых градиентов Флетчера-Ривза.

    Args:
        f: Функция для минимизации
        grad_f: Градиент функции
        x0: Начальная точка
        eps1: Точность по градиенту
        eps2: Точность по изменению функции
        max_iter: Максимальное число итераций

    Returns:
        Словарь с результатами оптимизации
    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'grad_norm': [np.linalg.norm(grad_f(x0))]
    }
    x = x0.copy()
    n = len(x0)

    grad = grad_f(x)
    d = -grad

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        # Критерий остановки
        if grad_norm < eps1:
            break

        # Перезапуск каждые n шагов
        if k > 0 and k % n == 0:
            d = -grad

        # Поиск шага с ограничением
        alpha = line_search(f, x, d)
        alpha = min(alpha, 1.0)

        # Новый пункт
        x_new = x + alpha * d

        # Проверка на NaN/Inf
        if np.any(np.isnan(x_new)) or np.any(np.isinf(x_new)):
            alpha = alpha * 0.5
            x_new = x + alpha * d
            if np.any(np.isnan(x_new)) or
np.any(np.isinf(x_new)):

```

```

        d = -grad
        alpha = 0.01
        x_new = x + alpha * d

    grad_new = grad_f(x_new)

    # Проверка градиента
    if np.any(np.isnan(grad_new)) or
np.any(np.isinf(grad_new)):
        grad_new = grad_f(x_new + 1e-8)

    # Проверка критериев останковки
    if (np.linalg.norm(x_new - x) < eps1 and
        abs(f(x_new) - f(x)) < eps2):
        x = x_new
        history['x'].append(x.copy())
        history['f'].append(f(x))
        history['grad_norm'].append(np.linalg.norm(grad_f(x)
))
        break

    # Вычисление w по формуле Флетчера-Ривза
    grad_norm_sq = np.linalg.norm(grad)**2
    if grad_norm_sq > 1e-20:
        w = np.linalg.norm(grad_new)**2 / grad_norm_sq
    else:
        w = 0

    # Ограничение w
    w = np.clip(w, 0, 1)

    # Новое направление
    d = -grad_new + w * d

    # Проверка направления спуска
    if np.dot(d, grad_new) > 0:
        d = -grad_new

    x = x_new
    grad = grad_new
    history['x'].append(x.copy())
    history['f'].append(f(x))
    history['grad_norm'].append(np.linalg.norm(grad))

    return {
        'x_star': x,
        'f_star': f(x),
        'iterations': len(history['f']) - 1,
        'history': history
    }

def polak_ribiere(f: Callable, grad_f: Callable, x0: np.ndarray,
                  eps1: float = DEFAULT_EPS1, eps2: float =

```

```

DEFAULT_EPS2,
                                max_iter: int = DEFAULT_MAX_ITER) -> Dict:
    """
    Метод сопряжённых градиентов Полака-Рибьера.

    Args:
        f: Функция для минимизации
        grad_f: Градиент функции
        x0: Начальная точка
        eps1: Точность по градиенту
        eps2: Точность по изменению функции
        max_iter: Максимальное число итераций

    Returns:
        Словарь с результатами оптимизации
    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'grad_norm': [np.linalg.norm(grad_f(x0))]
    }
    x = x0.copy()
    n = len(x0)

    grad = grad_f(x)
    d = -grad
    grad_prev = grad.copy()

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        # Критерий остановки
        if grad_norm < eps1:
            break

        # Поиск шага
        alpha = line_search(f, x, d)

        # Новый пункт
        x_new = x + alpha * d
        grad_new = grad_f(x_new)

        # Проверка критериев остановки
        if (np.linalg.norm(x_new - x) < eps1 and
            abs(f(x_new) - f(x)) < eps2):
            x = x_new
            history['x'].append(x.copy())
            history['f'].append(f(x))
            history['grad_norm'].append(np.linalg.norm(grad_f(x)))
    ))

    break

    # Вычисление w по формуле Полака-Рибьера

```

```

grad_diff = grad_new - grad_prev
grad_prev_norm_sq = np.linalg.norm(grad_prev)**2

if grad_prev_norm_sq > 1e-20:
    w = np.dot(grad_new, grad_diff) / grad_prev_norm_sq
else:
    w = 0

# Перезапуск каждые n шагов
if (k + 1) % n == 0:
    w = 0

# Ограничение w
w = np.clip(w, 0, 1)

# Новое направление
d = -grad_new + w * d

# Проверка направления спуска
if np.dot(d, grad_new) > 0:
    d = -grad_new

x = x_new
grad_prev = grad.copy()
grad = grad_new
history['x'].append(x.copy())
history['f'].append(f(x))
history['grad_norm'].append(np.linalg.norm(grad))

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': len(history['f']) - 1,
    'history': history
}

```

functions.py

```

"""
Модуль с тестовыми функциями и их градиентами
"""

import numpy as np
from typing import Callable

# Параметры по умолчанию для функции Розенброка
DEFAULT_A = 150
DEFAULT_B = 2
DEFAULT_F0 = 100

def rosenbrock(x: np.ndarray, a: float = DEFAULT_A, b: float =
DEFAULT_B,
              f0: float = DEFAULT_F0) -> float:

```

```

"""
Функция Розенброка для произвольной размерности n.


$$f(x) = \sum_{i=1}^{n-1} [a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2] + f_0$$


Args:
    x: Вектор аргументов
    a: Параметр a (по умолчанию 150)
    b: Параметр b (по умолчанию 2)
    f0: Свободный член (по умолчанию 100)

Returns:
    Значение функции в точке x
"""
n = len(x)
result = f0
for i in range(n - 1):
    result += a * (x[i]**2 - x[i+1])**2 + b * (x[i] - 1)**2
return result

def rosenbrock_gradient(x: np.ndarray, a: float = DEFAULT_A,
                        b: float = DEFAULT_B) -> np.ndarray:
    """
    Градиент функции Розенброка.

    Args:
        x: Вектор аргументов
        a: Параметр a
        b: Параметр b

    Returns:
        Вектор градиента в точке x
    """
    n = len(x)
    grad = np.zeros(n)

    for i in range(n - 1):
        grad[i] += 2 * a * (x[i]**2 - x[i+1]) * 2 * x[i] + 2 * b * (x[i] - 1)
        grad[i+1] += -2 * a * (x[i]**2 - x[i+1])

    return grad

def rosenbrock_hessian(x: np.ndarray, a: float = DEFAULT_A,
                       b: float = DEFAULT_B) -> np.ndarray:
    """
    Матрица Гессе функции Розенброка.

    Args:
        x: Вектор аргументов
        a: Параметр a
        b: Параметр b

```

```

Returns:
    Матрица Гессе в точке x
    """
    n = len(x)
    H = np.zeros((n, n))

    for i in range(n - 1):
        H[i, i] += 2 * a * (2 * x[i]**2 + 2 * (x[i]**2 -
x[i+1]))
        H[i, i+1] += -4 * a * x[i]
        H[i+1, i] += -4 * a * x[i]
        H[i+1, i+1] += 2 * a

    for i in range(n - 1):
        H[i, i] += 2 * b

    return H

def create_rosenbrock_function(a: float = DEFAULT_A, b: float =
DEFAULT_B,
                                f0: float = DEFAULT_F0) -> tuple:
    """
    Создать функцию Розенброка с заданными параметрами.

    Args:
        a: Параметр a
        b: Параметр b
        f0: Свободный член

    Returns:
        Кортеж (функция, градиент, гессиан)
    """
    def f(x):
        return rosenbrock(x, a, b, f0)

    def grad(x):
        return rosenbrock_gradient(x, a, b)

    def hess(x):
        return rosenbrock_hessian(x, a, b)

    return f, grad, hess

```

gradient_methods.py

```

"""
Модуль с методами градиентного спуска
"""

import numpy as np
from typing import Callable, Dict
from .line_search import line_search

```

```

# Параметры по умолчанию
DEFAULT_EPS1 = 1e-6
DEFAULT_EPS2 = 1e-8
DEFAULT_MAX_ITER = 1000

def gradient_descent(f: Callable, grad_f: Callable, x0:
np.ndarray,
                    eps1: float = DEFAULT_EPS1, eps2: float =
DEFAULT_EPS2,
                    max_iter: int = DEFAULT_MAX_ITER) -> Dict:
    """
    Метод наискорейшего градиентного спуска.

    Args:
        f: Функция для минимизации
        grad_f: Градиент функции
        x0: Начальная точка
        eps1: Точность по градиенту
        eps2: Точность по изменению функции
        max_iter: Максимальное число итераций

    Returns:
        Словарь с результатами:
        - x_star: найденная точка минимума
        - f_star: значение функции в точке минимума
        - iterations: число итераций
        - history: история сходимости
    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'grad_norm': [np.linalg.norm(grad_f(x0))]
    }
    x = x0.copy()

    for k in range(max_iter):
        grad = grad_f(x)
        grad_norm = np.linalg.norm(grad)

        # Критерий остановки по градиенту
        if grad_norm < eps1:
            break

        # Направление спуска
        d = -grad

        # Поиск шага
        alpha = line_search(f, x, d)

        # Новый пункт
        x_new = x + alpha * d

        # Проверка критериев остановки

```



```

        if (np.linalg.norm(x_new - x) < eps1 and
            abs(f(x_new) - f(x)) < eps2):
            x = x_new
            history['x'].append(x.copy())
            history['f'].append(f(x))
            history['grad_norm'].append(np.linalg.norm(grad_f(x))
    ))

        break

    x = x_new
    history['x'].append(x.copy())
    history['f'].append(f(x))
    history['grad_norm'].append(np.linalg.norm(grad_f(x)))

    return {
        'x_star': x,
        'f_star': f(x),
        'iterations': len(history['f']) - 1,
        'history': history
    }

```

line_search.py

"""

Модуль с методами одномерного поиска для определения шага

"""

import numpy as np

from typing import Callable

```

def golden_section_search(f: Callable, a: float, b: float,
                           tol: float = 1e-8) -> float:

```

"""

Метод золотого сечения для одномерной минимизации.

Args:

f: Функция для минимизации
a: Левая граница интервала
b: Правая граница интервала
tol: Точность

Returns:

Приближённое значение точки минимума

"""

phi = (1 + np.sqrt(5)) / 2

c = b - (b - a) / phi

d = a + (b - a) / phi

while abs(b - a) > tol:

if f(c) < f(d):

b = d

d = c

```

        c = b - (b - a) / phi
    else:
        a = c
        c = d
        d = a + (b - a) / phi

    return (a + b) / 2

def line_search(f: Callable, x: np.ndarray, d: np.ndarray,
               alpha_max: float = 10.0, tol: float = 1e-8) ->
float:
    """
    Одномерный поиск шага alpha вдоль направления d.

    Args:
        f: Функция для минимизации
        x: Текущая точка
        d: Направление поиска
        alpha_max: Максимальное значение шага
        tol: Точность поиска

    Returns:
        Оптимальная величина шага alpha
    """
    def phi(alpha):
        return f(x + alpha * d)

    return golden_section_search(phi, 0.0, alpha_max, tol)

```

optimizer.py

```

"""
Модуль с классом Optimizer для удобного запуска оптимизации
"""

import numpy as np
import time
from typing import Callable, Dict, List, Tuple, Optional
from . import (
    gradient_descent,
    fletcher_reeves,
    polak_ribiere,
    dfp,
    bfgs,
    lbfgs
)

class Optimizer:
    """
    Класс для выполнения оптимизации различными методами.
    """

    def __init__(self, f: Callable, grad_f: Callable, x0:

```

```

np.ndarray,
                                eps1: float = 1e-6, eps2: float = 1e-8,
max_iter: int = 1000):
    """
    Инициализация оптимизатора.

    Args:
        f: Функция для минимизации
        grad_f: Градиент функции
        x0: Начальная точка
        eps1: Точность по градиенту
        eps2: Точность по изменению функции
        max_iter: Максимальное число итераций
    """
    self.f = f
    self.grad_f = grad_f
    self.x0 = x0
    self.eps1 = eps1
    self.eps2 = eps2
    self.max_iter = max_iter
    self.results: Dict[str, Dict] = {}

    def run_all_methods(self, verbose: bool = True) -> Dict[str,
Dict]:
        """
        Запустить все методы оптимизации.

        Args:
            verbose: Выводить ли информацию о процессе

        Returns:
            Словарь с результатами всех методов
        """
        methods = [
            ('Градиентный спуск', gradient_descent),
            ('Флетчера-Ривза', fletcher_reeves),
            ('Полака-Рибьера', polak_ribiere),
            ('DFP', dfp),
            ('BFGS', bfgs),
            ('L-BFGS (m=10)', lambda f, g, x0, e1, e2, mi:
lbfgs(f, g, x0, e1, e2, mi, m=10))
        ]

        if verbose:
            print("\n" + "=" * 60)
            print("ЗАПУСК ВСЕХ МЕТОДОВ ОПТИМИЗАЦИИ")
            print("=" * 60)

        for name, method in methods:
            if verbose:
                print(f"\nЗапуск метода: {name}...")

            start_time = time.time()

```

```

        result = method(self.f, self.grad_f, self.x0,
                        self.eps1, self.eps2, self.max_iter)
        elapsed_time = time.time() - start_time

        self.results[name] = {
            **result,
            'time': elapsed_time
        }

        if verbose:
            print(f"    Найдено: x* = {result['x_star']}")
            print(f"    f(x*) = {result['f_star']:.8f}")
            print(f"    Итераций: {result['iterations']}")
            print(f"    Время: {elapsed_time:.4f} сек")

    return self.results

def run_method(self, method_name: str, **kwargs) -> Dict:
    """
    Запустить конкретный метод оптимизации.

    Args:
        method_name: Название метода
        **kwargs: Дополнительные параметры для метода

    Returns:
        Словарь с результатами
    """
    methods = {
        'gradient_descent': gradient_descent,
        'fletcher_reeves': fletcher_reeves,
        'polak_ribiere': polak_ribiere,
        'dfp': dfp,
        'bfgs': bfgs,
        'lbfgs': lbfgs
    }

    if method_name not in methods:
        raise ValueError(f"Неизвестный метод: {method_name}."

        f"Доступные: {list(methods.keys())}")

    method = methods[method_name]
    start_time = time.time()
    result = method(self.f, self.grad_f, self.x0,
                    self.eps1, self.eps2, self.max_iter,
    **kwargs)
    elapsed_time = time.time() - start_time

    self.results[method_name] = {
        **result,
        'time': elapsed_time
    }

```

```

        return self.results[method_name]

    def get_best_method(self, criterion: str = 'time') ->
    Tuple[str, Dict]:
        """
        Получить лучший метод по заданному критерию.

        Args:
            criterion: Критерий выбора ('time', 'iterations',
'accuracy')

        Returns:
            Кортеж (название метода, результаты)
        """
        if not self.results:
            raise ValueError("Сначала запустите методы
оптимизации")

        if criterion == 'time':
            best_name = min(self.results.keys(),
                            key=lambda k: self.results[k]['time'])
        elif criterion == 'iterations':
            best_name = min(self.results.keys(),
                            key=lambda k:
self.results[k]['iterations'])
        elif criterion == 'accuracy':
            best_name = min(self.results.keys(),
                            key=lambda k: np.linalg.norm(
                                self.grad_f(self.results[k]['x_sta
r'])))
        else:
            raise ValueError(f"Неизвестный критерий:
{criterion}")

        return best_name, self.results[best_name]

    def print_comparison_table(self):
        """
        Вывести сравнительную таблицу результатов.
        """
        if not self.results:
            print("Нет результатов для сравнения")
            return

        print("\n" + "=" * 80)
        print("СРАВНИТЕЛЬНАЯ ТАБЛИЦА МЕТОДОВ")
        print("=" * 80)
        print(f"{'Метод':<25} {'Итерации':<10} {'f(x*)':<15} "
              f"{'Время (сек)':<12} {'||∇f||':<12}")
        print("-" * 80)

        for name, result in self.results.items():

```

```

                                grad_norm      =
np.linalg.norm(self.grad_f(result['x_star']))
    print(f"{name:<25} {result['iterations']:<10} "
          f"{result['f_star']:<15.8f} "
          f"{result['time']:<12.4f} "
          f"{grad_norm:<12.2e}")

    print("=" * 80)

```

quasi_newton.py

```

"""
Модуль с квазиньютоновскими методами оптимизации
"""

import numpy as np
from typing import Callable, Dict, List
from .line_search import line_search

# Параметры по умолчанию
DEFAULT_EPS1 = 1e-6
DEFAULT_EPS2 = 1e-8
DEFAULT_MAX_ITER = 1000

def dfp(f: Callable, grad_f: Callable, x0: np.ndarray,
        eps1: float = DEFAULT_EPS1, eps2: float = DEFAULT_EPS2,
        max_iter: int = DEFAULT_MAX_ITER) -> Dict:
    """
    Квазиньютоновский метод Девидона-Флетчера-Пауэлла (DFP).

    Args:
        f: Функция для минимизации
        grad_f: Градиент функции
        x0: Начальная точка
        eps1: Точность по градиенту
        eps2: Точность по изменению функции
        max_iter: Максимальное число итераций

    Returns:
        Словарь с результатами оптимизации
    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'grad_norm': [np.linalg.norm(grad_f(x0))]
    }
    x = x0.copy()
    n = len(x0)

    G = np.eye(n) # Аппроксимация обратной матрицы Гессе
    grad = grad_f(x)

    for k in range(max_iter):

```

```

grad_norm = np.linalg.norm(grad)

# Критерий остановки
if grad_norm < eps1:
    break

# Направление поиска
d = -G @ grad

# Поиск шага
alpha = line_search(f, x, d)

# Новый пункт
x_new = x + alpha * d
grad_new = grad_f(x_new)

# Проверка критериев остановки
if (np.linalg.norm(x_new - x) < eps1 and
    abs(f(x_new) - f(x)) < eps2):
    x = x_new
    history['x'].append(x.copy())
    history['f'].append(f(x))
    history['grad_norm'].append(np.linalg.norm(grad_f(x)))
))

    break

# Обновление матрицы G (DFP формула)
delta_x = x_new - x
delta_g = grad_new - grad

dx_dg = np.dot(delta_x, delta_g)
if dx_dg > 1e-10:
    term1 = np.outer(delta_x, delta_x) / dx_dg
    Gg = G @ delta_g
    term2 = np.outer(Gg, Gg) / np.dot(delta_g, Gg)
    G = G + term1 - term2

x = x_new
grad = grad_new
history['x'].append(x.copy())
history['f'].append(f(x))
history['grad_norm'].append(np.linalg.norm(grad))

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': len(history['f']) - 1,
    'history': history
}

def bfgs(f: Callable, grad_f: Callable, x0: np.ndarray,
        eps1: float = DEFAULT_EPS1, eps2: float = DEFAULT_EPS2,
        max_iter: int = DEFAULT_MAX_ITER) -> Dict:

```

```
"""
    Квазиньютоновский метод BFGS (Бройдена-Флетчера-Гольдфарба-
    Шенно) .

```

```
Args:
```

```
    f: Функция для минимизации
    grad_f: Градиент функции
    x0: Начальная точка
    eps1: Точность по градиенту
    eps2: Точность по изменению функции
    max_iter: Максимальное число итераций

```

```
Returns:
```

```
    Словарь с результатами оптимизации

```

```
"""
```

```
history = {
    'x': [x0.copy()],
    'f': [f(x0)],
    'grad_norm': [np.linalg.norm(grad_f(x0))]
}
```

```
x = x0.copy()
```

```
n = len(x0)
```

```
H = np.eye(n) # Аппроксимация обратной матрицы Гессе
```

```
grad = grad_f(x)
```

```
for k in range(max_iter):
```

```
    grad_norm = np.linalg.norm(grad)
```

```
    # Критерий остановки
```

```
    if grad_norm < eps1:
```

```
        break
```

```
    # Направление поиска
```

```
    d = -H @ grad
```

```
    # Поиск шага
```

```
    alpha = line_search(f, x, d)
```

```
    # Новый пункт
```

```
    x_new = x + alpha * d
```

```
    grad_new = grad_f(x_new)
```

```
    # Проверка критериев остановки
```

```
    if (np.linalg.norm(x_new - x) < eps1 and
```

```
        abs(f(x_new) - f(x)) < eps2):
```

```
        x = x_new
```

```
        history['x'].append(x.copy())
```

```
        history['f'].append(f(x))
```

```
        history['grad_norm'].append(np.linalg.norm(grad_f(x)
```

```
    ))
```

```
    break
```



```

# Обновление матрицы H (BFGS формула)
s = x_new - x
y = grad_new - grad

sy = np.dot(s, y)
if sy > 1e-10:
    rho = 1.0 / sy
    I = np.eye(n)
    V = I - rho * np.outer(s, y)
    H = V @ H @ V.T + rho * np.outer(s, s)

x = x_new
grad = grad_new
history['x'].append(x.copy())
history['f'].append(f(x))
history['grad_norm'].append(np.linalg.norm(grad))

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': len(history['f']) - 1,
    'history': history
}

def lbfgs(f: Callable, grad_f: Callable, x0: np.ndarray,
          eps1: float = DEFAULT_EPS1, eps2: float =
DEFAULT_EPS2,
          max_iter: int = DEFAULT_MAX_ITER, m: int = 10) ->
Dict:
    """
    Квазиньютоновский метод L-BFGS (Limited-memory BFGS).

    Args:
        f: Функция для минимизации
        grad_f: Градиент функции
        x0: Начальная точка
        eps1: Точность по градиенту
        eps2: Точность по изменению функции
        max_iter: Максимальное число итераций
        m: Размер памяти (число сохраняемых пар векторов)

    Returns:
        Словарь с результатами оптимизации
    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'grad_norm': [np.linalg.norm(grad_f(x0))]
    }
    x = x0.copy()

    grad = grad_f(x)

```

```

# Хранилище для пар векторов (s, y) и коэффициентов rho
s_list: List[np.ndarray] = []
y_list: List[np.ndarray] = []
rho_list: List[float] = []

for k in range(max_iter):
    grad_norm = np.linalg.norm(grad)

    # Критерий остановки
    if grad_norm < eps1:
        break

    # Двухпетлевая рекурсия для вычисления направления
    q = grad.copy()
    alpha_list: List[float] = []

    # Первый цикл (обратный)
    for i in range(len(s_list) - 1, -1, -1):
        s = s_list[i]
        y = y_list[i]
        rho = rho_list[i]
        alpha_i = rho * np.dot(s, q)
        alpha_list.insert(0, alpha_i)
        q = q - alpha_i * y

    # Инициализация H_0
    if len(s_list) > 0:
        s_last = s_list[-1]
        y_last = y_list[-1]
        gamma = np.dot(s_last, y_last) / np.dot(y_last,
y_last)
    else:
        gamma = 1.0

    r = gamma * q

    # Второй цикл (прямой)
    for i in range(len(s_list)):
        s = s_list[i]
        y = y_list[i]
        rho = rho_list[i]
        alpha_i = alpha_list[i]
        beta = rho * np.dot(y, r)
        r = r + s * (alpha_i - beta)

    d = -r

    # Поиск шага
    alpha = line_search(f, x, d)

    # Новый пункт
    x_new = x + alpha * d
    grad_new = grad_f(x_new)

```

```

# Проверка критериев останавки
if (np.linalg.norm(x_new - x) < eps1 and
    abs(f(x_new) - f(x)) < eps2):
    x = x_new
    history['x'].append(x.copy())
    history['f'].append(f(x))
    history['grad_norm'].append(np.linalg.norm(grad_f(x)
))
        break

# Сохранение новой пары векторов
s = x_new - x
y = grad_new - grad

sy = np.dot(s, y)
if sy > 1e-10:
    if len(s_list) >= m:
        s_list.pop(0)
        y_list.pop(0)
        rho_list.pop(0)

    s_list.append(s)
    y_list.append(y)
    rho_list.append(1.0 / sy)

x = x_new
grad = grad_new
history['x'].append(x.copy())
history['f'].append(f(x))
history['grad_norm'].append(np.linalg.norm(grad))

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': len(history['f']) - 1,
    'history': history
}

```

visualization.py

```

"""
Модуль визуализации результатов оптимизации
"""

import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm
from matplotlib.colors import LogNorm
from mpl_toolkits.mplot3d import Axes3D
from typing import Dict, List, Callable, Optional
import os

```

```

def plot_convergence(results: Dict, f_true: float = 100,
                    save_path: Optional[str] = None):
    """
    Построить графики сходимости методов с разделением на
    быстрые/медленные.

    Args:
        results: Словарь с результатами методов
        f_true: Истинное значение минимума функции
        save_path: Путь для сохранения графика (опционально)
    """
    # Разделяем методы на быстрые (< 100 итераций) и медленные
    fast_methods = {}
    slow_methods = {}

    for name, result in results.items():
        if result['iterations'] <= 100:
            fast_methods[name] = result
        else:
            slow_methods[name] = result

    # Цвета для методов
    all_names = list(results.keys())
    colors_map = dict(zip(all_names,
                          cm.get_cmap('tab10')(np.linspace(0, 1, len(all_names)))))

    # Создаём два отдельных графика
    if fast_methods:
        _plot_convergence_group(fast_methods, f_true, "fast",
                                colors_map, save_path)

    if slow_methods:
        _plot_convergence_group(slow_methods, f_true, "slow",
                                colors_map, save_path)

    # Также создаём сводный график с нормализацией
    _plot_summary_comparison(results, f_true, save_path)

def _plot_convergence_group(results: Dict, f_true: float,
                            group_name: str,
                            colors_map: dict, save_path:
                            Optional[str] = None):
    """
    Построить графики для группы методов.
    """
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    method_names = list(results.keys())
    colors = [colors_map[name] for name in method_names]

    # 1. Сходимость по функции
    ax = axes[0, 0]
    for name, result in results.items():

```

```

        f_history = np.array(result['history']['f'])
        iterations = range(len(f_history))
        ax.plot(iterations, f_history, label=name, linewidth=2,
color=colors_map[name])
        ax.set_xlabel('Итерация')
        ax.set_ylabel('f(x)')
        ax.set_title(f'Сходимость по значению функции ({"быстрые" if
group_name == "fast" else "медленные"} методы)')
        ax.legend(fontsize=8)
        ax.grid(True, alpha=0.3)

# 2. Сходимость по градиенту
ax = axes[0, 1]
for name, result in results.items():
    grad_history = np.array(result['history']['grad_norm'])
    iterations = range(len(grad_history))
    ax.semilogy(iterations, grad_history, label=name,
linewidth=2, color=colors_map[name])
    ax.set_xlabel('Итерация')
    ax.set_ylabel('|| $\nabla f(x)$ ||')
    ax.set_title(f'Сходимость по норме градиента ({"быстрые" if
group_name == "fast" else "медленные"} методы)')
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)

# 3. Время выполнения
ax = axes[1, 0]
times = [results[name]['time'] for name in method_names]
bars = ax.bar(method_names, times, color=[colors_map[name]
for name in method_names], alpha=0.8)
ax.set_xlabel('Метод')
ax.set_ylabel('Время (сек)')
ax.set_title('Время выполнения')
ax.tick_params(axis='x', rotation=45)

for bar, val in zip(bars, times):
    ax.text(bar.get_x() + bar.get_width()/2,
bar.get_height() + 0.001,
f'{val:.3f}c', ha='center', va='bottom',
fontsize=9)

# 4. Число итераций
ax = axes[1, 1]
iterations_list = [results[name]['iterations'] for name in
method_names]
bars = ax.bar(method_names, iterations_list,
color=[colors_map[name] for name in method_names], alpha=0.8)
ax.set_ylabel('Итерации')
ax.set_title('Число итераций')
ax.tick_params(axis='x', rotation=45)

for bar, val in zip(bars, iterations_list):
    ax.text(bar.get_x() + bar.get_width()/2,

```

```

bar.get_height() + 0.5,
        str(val), ha='center', va='bottom', fontsize=9)

plt.tight_layout()

if save_path:
    base_path = save_path.rsplit('.', 1)[0]
    new_path = f"{base_path}_{group_name}.png"
    plt.savefig(new_path, dpi=150, bbox_inches='tight')
    print(f"График для {group_name} методов сохранён в
'{new_path}'")

plt.show()

def __plot_summary_comparison(results: Dict, f_true: float,
save_path: Optional[str] = None):
    """
    Построить сводный сравнительный график (нормализованный).
    """
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    method_names = list(results.keys())
    colors = list(cm.get_cmap('tab10')(np.linspace(0, 1,
len(method_names))))

    # Нормализация
    all_times = [results[name]['time'] for name in method_names]
    all_iters = [results[name]['iterations'] for name in
method_names]

    time_max = max(all_times)
    iter_max = max(all_iters)

    # 1. Время (норм.)
    ax = axes[0]
    times_norm = [t / time_max for t in all_times]
    bars = ax.bar(method_names, times_norm, color=colors,
alpha=0.8)
    ax.set_ylabel('Время (норм.)')
    ax.set_title('Время выполнения (нормированное)')
    ax.tick_params(axis='x', rotation=45)

    for bar, norm_val, raw_val in zip(bars, times_norm,
all_times):
        ax.text(bar.get_x() + bar.get_width()/2,
bar.get_height() + 0.02,
f'{norm_val:.2f}\n({raw_val:.3f}c)',
ha='center', va='bottom', fontsize=8)

    # 2. Итерации (норм.)
    ax = axes[1]
    iters_norm = [i / iter_max for i in all_iters]
    bars = ax.bar(method_names, iters_norm, color=colors,

```

```

alpha=0.8)
    ax.set_ylabel('Итерации (норм.)')
    ax.set_title('Число итераций (нормированное)')
    ax.tick_params(axis='x', rotation=45)

    for bar, norm_val, raw_val in zip(bars, iters_norm,
all_iters):
        ax.text(bar.get_x() + bar.get_width()/2,
bar.get_height() + 0.02,
        f'{norm_val:.2f}\n({raw_val})', ha='center',
va='bottom', fontsize=8)

    # 3. Точность (норм.) - обратная величина
    ax = axes[2]

    grad_norms =
[ np.linalg.norm([results[name]['history']['grad_norm'][-1]]) for
name in method_names]
    grad_max = max(grad_norms)
    grad_norms_norm = [g / grad_max for g in grad_norms]
    bars = ax.bar(method_names, grad_norms_norm, color=colors,
alpha=0.8)
    ax.set_ylabel('|| $\nabla f$ || (норм.)')
    ax.set_title('Точность (нормированная)')
    ax.tick_params(axis='x', rotation=45)

    for bar, norm_val, raw_val in zip(bars, grad_norms_norm,
grad_norms):
        ax.text(bar.get_x() + bar.get_width()/2,
bar.get_height() + 0.02,
        f'{norm_val:.2f}\n({raw_val:.1e})', ha='center',
va='bottom', fontsize=7)

plt.tight_layout()

if save_path:
    base_path = save_path.rsplit('.', 1)[0]
    new_path = f'{base_path}_summary.png'
    plt.savefig(new_path, dpi=150, bbox_inches='tight')
    print(f"Сводный график сохранён в '{new_path}'")

plt.show()

def plot_trajectories(results: Dict, f_2d: Callable,
                    x0: np.ndarray, x_true: np.ndarray,
                    f_true: float = 100,
                    save_path: Optional[str] = None):
    """
    Построить траектории методов на поверхности функции.

    Args:
        results: Словарь с результатами методов
        f_2d: 2D функция для визуализации
        x0: Начальная точка

```

```

x_true: Истинная точка минимума
f_true: Истинное значение минимума
save_path: Путь для сохранения графика (опционально)
"""
# Создание сетки для 2D проекции
x1 = np.linspace(-0.5, 1.5, 100)
x2 = np.linspace(-0.5, 1.5, 100)
X1, X2 = np.meshgrid(x1, x2)
Z = np.zeros_like(X1)

for i in range(len(x1)):
    for j in range(len(x2)):
        Z[j, i] = f_2d(x1[i], x2[j])

fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# 1. Поверхность функции
ax = axes[0, 0]
contour = ax.contourf(X1, X2, Z, levels=50, cmap='viridis',
alpha=0.8)
ax.plot(x_true[0], x_true[1], 'r*', markersize=15,
label='Глобальный минимум')
ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_title('Поверхность функции (проекция x1-x2)')
ax.legend()
plt.colorbar(contour, ax=ax)

# 2. Траектории всех методов (проецируем 3D на 2D)
ax = axes[0, 1]
contour = ax.contourf(X1, X2, Z, levels=50, cmap='viridis',
alpha=0.6)

colors = ['red', 'blue', 'green', 'orange', 'purple',
'brown']
for (name, result), color in zip(results.items(), colors):
    x_path = np.array(result['history']['x'])
    # Проецируем 3D траекторию на 2D (первые две координаты)
    ax.plot(x_path[:, 0], x_path[:, 1], 'o-', color=color,
label=name, markersize=3, linewidth=1.5,
alpha=0.7)

ax.plot(x_true[0], x_true[1], 'r*', markersize=15,
label='Минимум')
ax.plot(x0[0], x0[1], 'ks', markersize=10, label='Старт')
ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_title('Траектории методов оптимизации')
ax.legend(fontsize=7, loc='upper right')

# 3. Линии уровня с траекторией лучшего метода
ax = axes[0, 2]
contour = ax.contour(X1, X2, Z, levels=30, cmap='viridis')

```



```

# Найти лучший метод по итерациям
best_name = min(results.keys(), key=lambda k:
results[k]['iterations'])
x_path = np.array(results[best_name]['history']['x'])
ax.plot(x_path[:, 0], x_path[:, 1], 'ro-', markersize=4,
linewidth=2,
        label=f'Лучший: {best_name}')
ax.plot(x_true[0], x_true[1], 'r*', markersize=15)
ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_title(f'Траектория {best_name}')
ax.legend()

# 4. Сходимость по функции (лог)
ax = axes[1, 0]
for (name, result), color in zip(results.items(), colors):
    f_history = np.array(result['history']['f'])
    f_rel = f_history - f_true + 1e-10
    ax.semilogy(range(len(f_history)), f_rel, label=name,
                linewidth=2, color=color)
ax.set_xlabel('Итерация')
ax.set_ylabel('f(x) - f* (лог)')
ax.set_title('Сходимость по значению функции')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)

# 5. Сходимость по градиенту
ax = axes[1, 1]
for (name, result), color in zip(results.items(), colors):
    grad_history = np.array(result['history']['grad_norm'])
    ax.semilogy(range(len(grad_history)), grad_history,
                label=name, linewidth=2, color=color)
ax.set_xlabel('Итерация')
ax.set_ylabel('||∇f(x)|| (лог)')
ax.set_title('Сходимость по норме градиента')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)

# 6. Гистограмма итераций
ax = axes[1, 2]
names = list(results.keys())
iterations = [results[name]['iterations'] for name in names]
bars = ax.bar(names, iterations, color=colors[:len(names)],
alpha=0.8)
ax.set_ylabel('Число итераций')
ax.set_title('Число итераций до сходимости')
ax.tick_params(axis='x', rotation=45)

for bar, val in zip(bars, iterations):
    ax.text(bar.get_x() + bar.get_width()/2,
bar.get_height() + 1,
            str(val), ha='center', va='bottom', fontsize=10)

```

```

plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    print(f"Траектории сохранены в '{save_path}'")

plt.show()

def plot_all_trajectories(results: Dict, f_2d: Callable,
                        x0: np.ndarray, x_true: np.ndarray,
                        save_path: Optional[str] = None):
    """
    Построить все траектории методов на одном графике с линиями
    уровня.

    Args:
        results: Словарь с результатами методов
        f_2d: 2D функция для визуализации
        x0: Начальная точка
        x_true: Истинная точка минимума
        save_path: Путь для сохранения графика (опционально)
    """
    # Создание сетки для 2D проекции
    x1 = np.linspace(-0.5, 1.5, 150)
    x2 = np.linspace(-0.5, 1.5, 150)
    X1, X2 = np.meshgrid(x1, x2)
    Z = np.zeros_like(X1)

    for i in range(len(x1)):
        for j in range(len(x2)):
            Z[j, i] = f_2d(x1[i], x2[j])

    fig, axes = plt.subplots(1, 1, figsize=(12, 10))

    # Линии уровня
    contour = axes.contourf(X1, X2, Z, levels=50,
                           cmap='viridis', alpha=0.8)
    contour_lines = axes.contour(X1, X2, Z, levels=20,
                                 colors='white',
                                 linewidths=0.5, alpha=0.5)

    # Цвета для методов
    method_names = list(results.keys())
    colors = list(cm.get_cmap('tab10')(np.linspace(0, 1,
len(method_names))))

    # Траектории всех методов
    for (name, result), color in zip(results.items(), colors):
        x_path = np.array(result['history']['x'])
        # Проецируем 3D траекторию на 2D (первые две координаты)
        axes.plot(x_path[:, 0], x_path[:, 1], 'o-', color=color,
                  label=name, markersize=2, linewidth=1,

```

```

alpha=0.7)

    # Отмечаем начало и конец траектории
    axes.plot(x_path[0, 0], x_path[0, 1], 's', color=color,
markersize=8,
               alpha=0.5) # Старт
    axes.plot(x_path[-1, 0], x_path[-1, 1], '*',
color=color, markersize=12,
               alpha=0.8) # Финиш

    # Глобальный минимум
    axes.plot(x_true[0], x_true[1], 'r*', markersize=20,
label='Глобальный минимум (1,1)',
           zorder=10)

    # Начальная точка
    axes.plot(x0[0], x0[1], 'ks', markersize=12, label='Старт
(0,0)', zorder=10)

    axes.set_xlabel('x1', fontsize=12)
    axes.set_ylabel('x2', fontsize=12)
    axes.set_title('Все траектории методов
оптимизации\n(проекция на плоскость x1-x2)',
           fontsize=14)
    axes.legend(loc='upper right', fontsize=8,
bbox_to_anchor=(1.35, 1.05))

    # Добавляем цветовую шкалу
    cbar = plt.colorbar(contour, ax=axes)
    cbar.set_label('f(x)', fontsize=10)

    plt.tight_layout()

    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
        print(f"Все траектории сохранены в '{save_path}'")

    plt.show()

def plot_individual_trajectories(results: Dict, f_2d: Callable,
                                x0: np.ndarray, x_true:
                                np.ndarray,
                                save_path: Optional[str] =
None):
    """
    Построить отдельные графики траекторий для каждого метода.

    Args:
        results: Словарь с результатами методов
        f_2d: 2D функция для визуализации
        x0: Начальная точка
        x_true: Истинная точка минимума
        save_path: Путь для сохранения графиков (опционально)

```

```

"""
# Создание сетки для 2D проекции
x1 = np.linspace(-0.5, 1.5, 150)
x2 = np.linspace(-0.5, 1.5, 150)
X1, X2 = np.meshgrid(x1, x2)
Z = np.zeros_like(X1)

for i in range(len(x1)):
    for j in range(len(x2)):
        Z[j, i] = f_2d(x1[i], x2[j])

method_names = list(results.keys())
colors = list(cm.get_cmap('tab10')(np.linspace(0, 1,
len(method_names))))

# Создаём отдельный график для каждого метода
for (name, result), color in zip(results.items(), colors):
    fig, ax = plt.subplots(1, 1, figsize=(10, 8))

    x_path = np.array(result['history']['x'])

    # Линии уровня
    contour = ax.contourf(X1, X2, Z, levels=50,
cmap='viridis', alpha=0.8)
    ax.contour(X1, X2, Z, levels=20, colors='white',
linewidths=0.5, alpha=0.5)

    # Траектория метода
    ax.plot(x_path[:, 0], x_path[:, 1], 'o-', color=color,
            markersize=3, linewidth=1.5, alpha=0.8,
label=f'{name}')

    # Отмечаем начало и конец
    ax.plot(x_path[0, 0], x_path[0, 1], 'gs', markersize=10,
            label='Старт', zorder=10)
    ax.plot(x_path[-1, 0], x_path[-1, 1], 'r*',
markersize=15,
            label='Финиш', zorder=10)

    # Глобальный минимум
    ax.plot(x_true[0], x_true[1], 'r*', markersize=20,
            label=f'Минимум ({x_true[0]}, {x_true[1]}),',
zorder=10)

    ax.set_xlabel('x1', fontsize=12)
    ax.set_ylabel('x2', fontsize=12)
    ax.set_title(f'Траектория: {name}\n'
                f'Итераций: {result["iterations"]}, '
                f'f(x*)={result["f_star"]:.8f}, '
                f'Время: {result["time"]:.4f}c',
                fontsize=12)
    ax.legend(loc='upper right', fontsize=9)

```

```

# Цветовая шкала
cbar = plt.colorbar(contour, ax=ax)
cbar.set_label('f(x)', fontsize=10)

plt.tight_layout()

if save_path:
    # Сохраняем каждый график отдельно
    base_path = save_path.rsplit('.', 1)[0]
    # Создаём безопасное имя файла
    safe_name = name.replace(' ', '_').replace('(',
    ').replace(')', '_').replace('-', '_')
    new_path = f"{base_path}_{safe_name}.png"
    plt.savefig(new_path, dpi=150, bbox_inches='tight')
    print(f"Траектория '{name}' сохранена в '{new_path}'")

plt.show()

def plot_3d_surface(f_2d: Callable, x_true: np.ndarray,
                    save_path: Optional[str] = None):
    """
    3D визуализация функции.

    Args:
        f_2d: 2D функция для визуализации
        x_true: Истинная точка минимума
        save_path: Путь для сохранения графика (опционально)
    """
    fig = plt.figure(figsize=(14, 6))

    # 3D поверхность
    ax1 = fig.add_subplot(121, projection='3d')

    x1 = np.linspace(-0.5, 1.5, 80)
    x2 = np.linspace(-0.5, 1.5, 80)
    X1, X2 = np.meshgrid(x1, x2)
    Z = np.zeros_like(X1)

    for i in range(len(x1)):
        for j in range(len(x2)):
            Z[j, i] = f_2d(x1[i], x2[j])

    ax1.plot_surface(X1, X2, Z, cmap='viridis', alpha=0.9,
                     edgecolor='none')
    ax1.set_xlabel('x1')
    ax1.set_ylabel('x2')
    ax1.set_zlabel('f(x)')
    ax1.set_title('3D поверхность функции')

    # 3D контур
    ax2 = fig.add_subplot(122, projection='3d')
    ax2.contour3D(X1, X2, Z, 50, cmap='viridis')

```

```
ax2.set_xlabel('x1')
ax2.set_ylabel('x2')
ax2.set_zlabel('f(x)')
ax2.set_title('3D контуры функции')

plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    print(f"3D визуализация сохранена в '{save_path}'")

plt.show()

def create_visualization(results: Dict, f_2d: Callable,
                        x0: np.ndarray, x_true: np.ndarray,
                        f_true: float = 100,
                        output_dir: str = '.'):
    """
    Создать все визуализации.

    Args:
        results: Словарь с результатами методов
        f_2d: 2D функция для визуализации
        x0: Начальная точка
        x_true: Истинная точка минимума
        f_true: Истинное значение минимума
        output_dir: Директория для сохранения графиков
    """
    # Создаем директорию для графиков если не существует
    plots_dir = os.path.join(output_dir, 'plots')
    os.makedirs(plots_dir, exist_ok=True)

    print("\n" + "=" * 60)
    print("ГЕНЕРАЦИЯ ВИЗУАЛИЗАЦИИ")
    print("=" * 60)

    # 1. Графики сходимости (разделённые на быстрые/медленные + сводный)
    plot_convergence(
        results,
        f_true,
        save_path=os.path.join(plots_dir, 'convergence.png')
    )

    # 2. Все траектории на одном графике
    plot_all_trajectories(
        results,
        f_2d,
        x0,
        x_true,
        save_path=os.path.join(plots_dir,
'all_trajectories.png')
    )
```

```

# 3. Отдельные траектории для каждого метода (6 графиков)
plot_individual_trajectories(
    results,
    f_2d,
    x0,
    x_true,
    save_path=os.path.join(plots_dir, 'trajectory.png')
)

# 4. Детальные траектории (сетка 2x3)
plot_trajectories(
    results,
    f_2d,
    x0,
    x_true,
    f_true,
    save_path=os.path.join(plots_dir,
'trajectories_grid.png')
)

# 5. 3D поверхность
plot_3d_surface(
    f_2d,
    x_true,
    save_path=os.path.join(plots_dir, 'surface_3d.png')
)

print(f"\nВсе графики сохранены в директорию
'{plots_dir}/'")

```

Вывод

В ходе выполнения лабораторной работы изучили алгоритмы многомерного поиска 1-го и 2-го порядка, разрабатывали программы реализации алгоритмов многомерного поиска 1-го и 2-го порядка, вычисляли экстремумы функции.